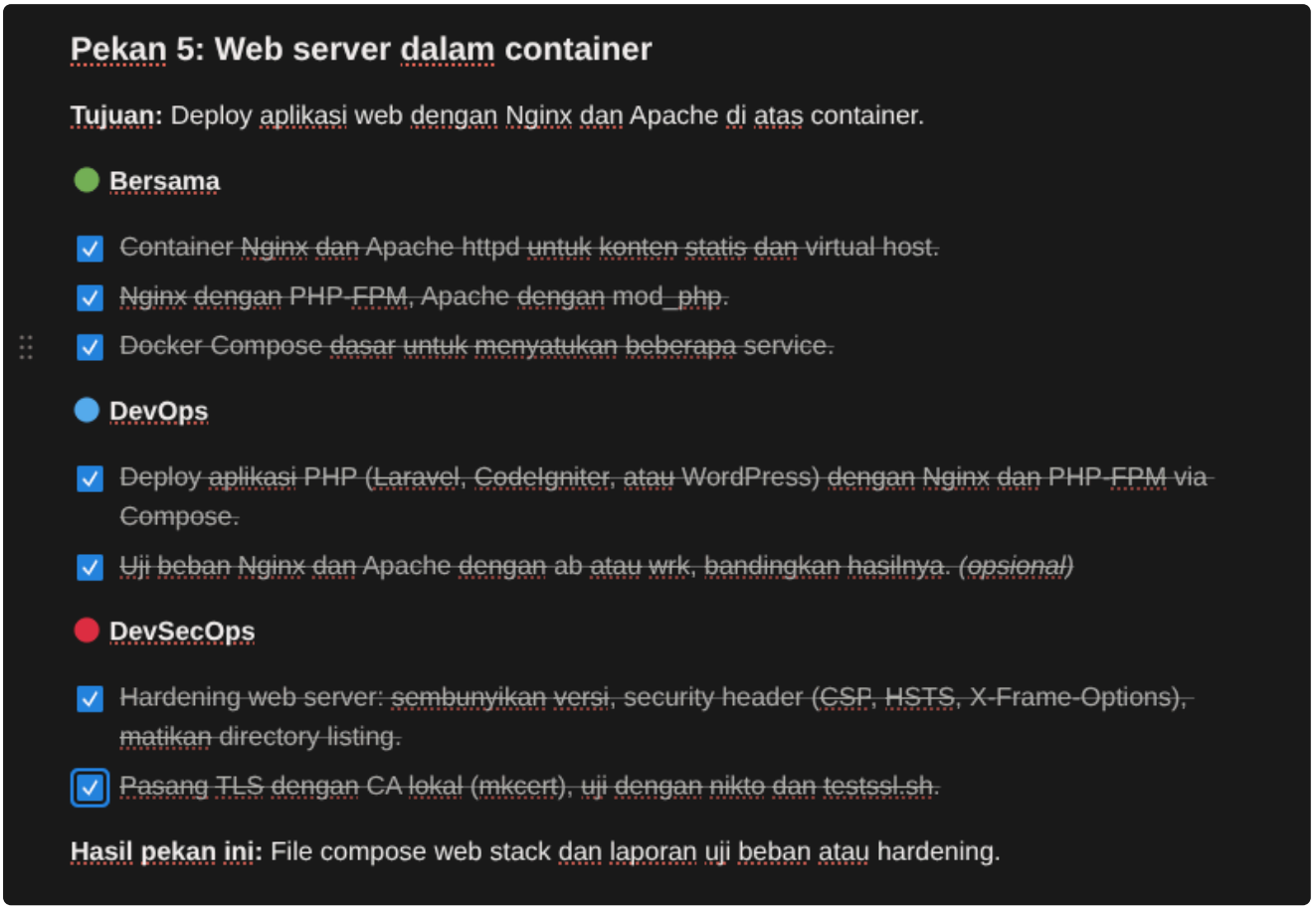
Dokumentasi task 5

Praktik Web Server dengan Docker

Ringkasan Isi Catatan praktik hari ini, mencakup:

1. Konten statis dan virtual host
2. Nginx + PHP-FPM dan Apache + mod_php
3. Docker Compose Dasar
4. Deploy WordPress via Compose
5. Uji Beban Apache dan Nginx
6. Hardening Nginx
7. TLS dengan CA Lokal

disini aku akan menulis apa yang aku praktikkan yang mana ada beberapa hal yang aku praktikkan pada hari ini yang mana itu adalah



Checklist 1 Konten Statis dan Virtual Host

langsung aja kita akan mencoba container nginx dan apache untuk konten statis dan virtual host dan sebelumnya aku akan menjelaskan sedikit soal apa itu konten statis dan apa itu virtual host,

> ⓘ **Konten Statis konten statis itu dimana sebuah web server mengirim file yang apa adanya tanpa proses eksekusi dibaliknya, kaya web statis biasa gitu cuma html css yang mana berbeda dengan php karena php butuh interpreter untuk generate output,**

> ⓘ **Virtual Host lalu virtual host yang mana disebut server block pada nginx adalah kemampuan dimana sebuah web server itu dapat melayani beberapa website atau domain yang berbeda sekaligus yang mana itu dibedakan dengan nama domain yang diminta ato nama domain yang aku mau dan pembedanya nanti juga ada pada header host pada request http/https**

Konten Statis dengan Apache

langsung aja kita akan membuat sebuah server block aja karena kalau konten statis mah bisa hanya dengan jalankan command

```
sudo apt install apache2
```

terus

```
sudo systemctl enabe apache2
```

terus

```
cd /var/www/html
```

terus

```
echo "tes ini html" > index.html
```

terus tinggal

```
localhost:80
```

keluar tuh hasilnya dan kita akan mulai membuat server block dan virtual host aja sekaligus yang mana kita akan mulai dengan server block,

Server Block pada Nginx

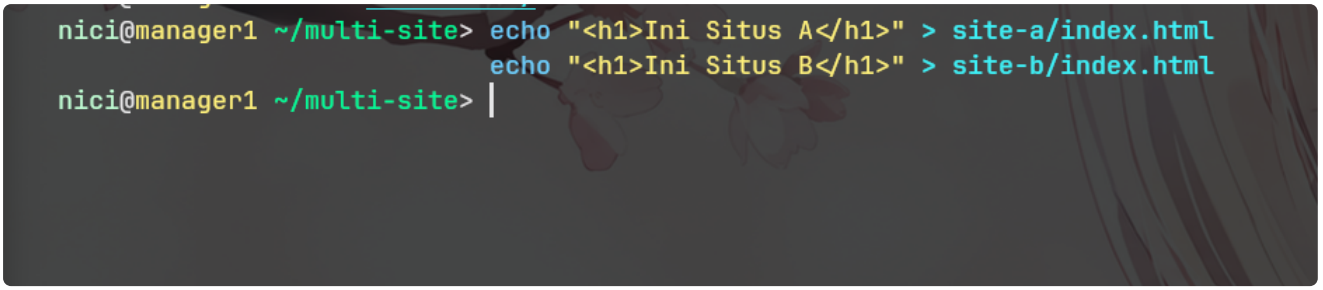
kita mulai dari langsung membuat konfigurasi nginx dengan command..sebelum itu bikin direktori dulu dengan command

```
mkdir -p ~/multi-site/site-a ~/multi-site/site-b
```

lalu

```
cd ~/multisite
```

terus buat dulu index.html masing masing nya dengan command



baru kita langsung buat konfigurasinya dengan command

```
nano nginx-vhost.conf
```

lalu isi dengan ini

```
server {
    listen 80;
    server_name site-a.local;
    root /sites/site-a;
    index index.html;
}

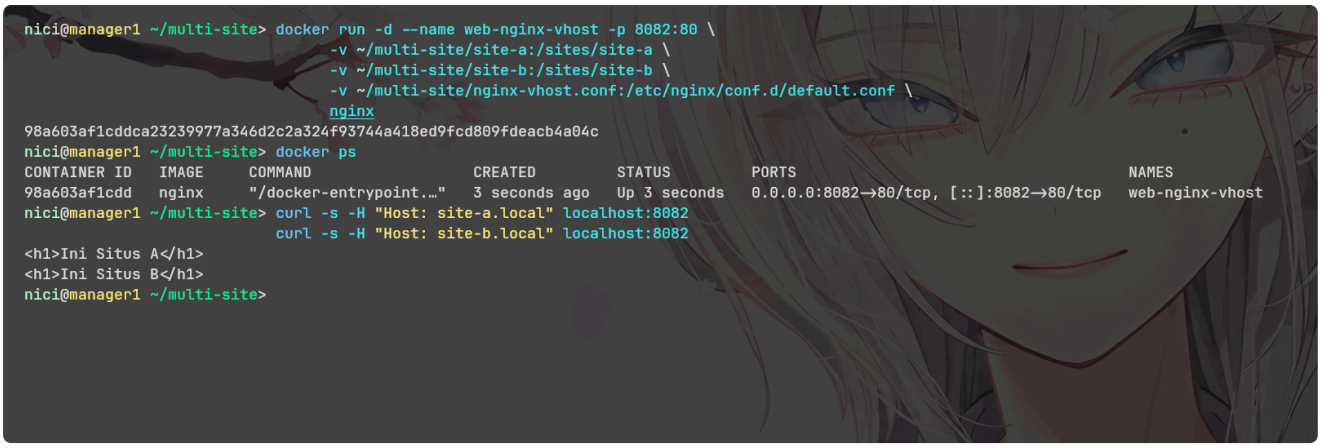
server {
    listen 80;
    server_name site-b.local;
    root /sites/site-b;
    index index.html;
}
```

udah tuh dan disitu kalau kita baca harusnya kita sudah sadar soal pembeda antara domain a dan b adalah server_name yang ada pada konfigurasi dan langsung aja eksekusi dengan command

```
docker run -d --name web-nginx-vhost -p 8082:80 \
  -v ~/multi-site/site-a:/sites/site-a \
  -v ~/multi-site/site-b:/sites/site-b \
```

```
    -v ~/multi-site/nginx-vhost.conf:/etc/nginx/conf.d/default.conf \
    nginx
```

yang hasilnya adalah

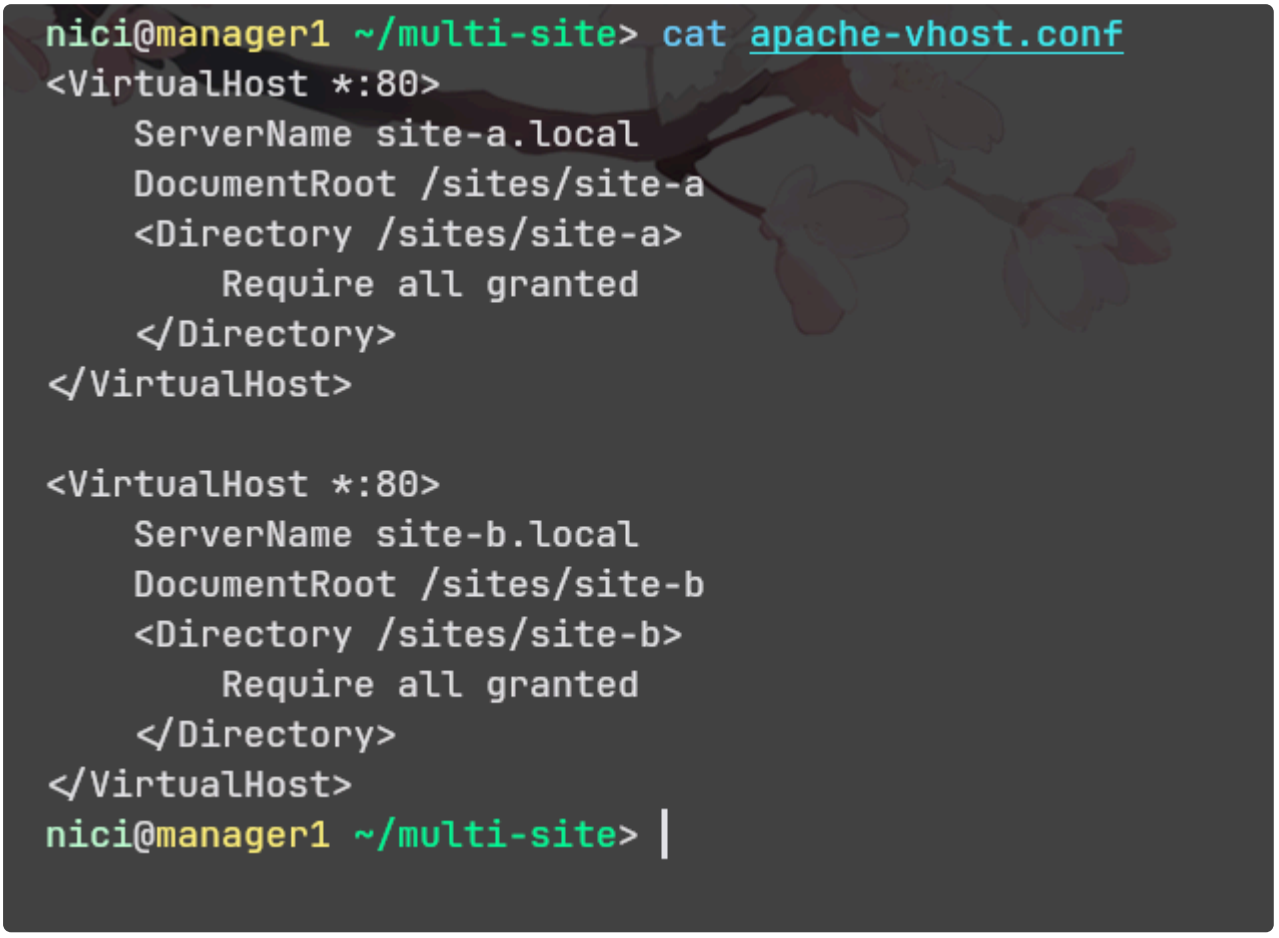


Virtual Host pada Apache

oke jadi itu untuk server block, dan bagaimana dengan virtual host pada apache? sama aja bosku dan kita langsung bikina aja konfigurasinya di dalam direktori ~/multisite menggunakan nano yang command nya adalah

```
nano apache-vhost.conf
```

terus isi deh sama ini



lalu tinggal eksekusi dengan command ini

```
docker run -d --name web-apache-vhost -p 8083:80 \
-v ~/multi-site/site-a:/sites/site-a \
-v ~/multi-site/site-b:/sites/site-b \
-v ~/multi-site/apache-vhost.conf:/usr/local/apache2/conf/extra/httpd-
vhosts.conf \ httpd
```

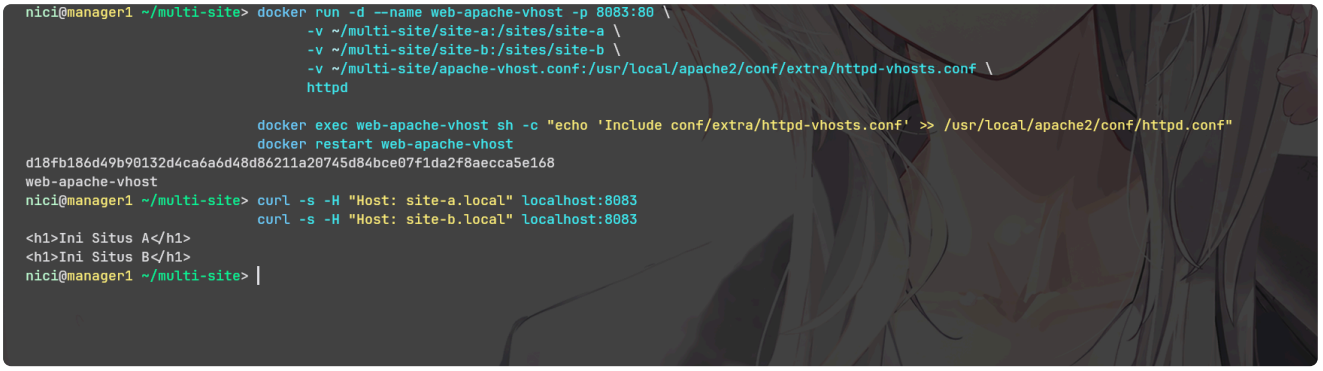
terus tinggal di exec pake command

```
docker exec web-apache-vhost sh -c "echo 'Include conf/extra/httpd-
vhosts.conf' >> /usr/local/apache2/conf/httpd.conf"
```

lalu restart

```
docker restart web-apache-vhost
```

terus cek hasilnya langsung pake curl



kira kira seperti itu bos

Checklist 2 Nginx + PHP-FPM dan Apache + mod_php

lalu kita lanjut ke checklist 2, yaitu nginx dengan php fpm dan apache dengan menggunakan modul php nya, seperti biasa aku akan menjelaskan secara singkat soal keduanya yang mana kenapa nginx perlu menggunakan php fpm? nah kita harus pahami ini dari php nya sendiri yang mana php itu dapat berjalan via cli jika kita membuat sebuah file yang misalkan namanya adalah tes.php yang isinya hanya echo "halo" gitu tok dan kita bisa menjalankannya dengan command

```
php tes.php
```

yang mana akan menghasilkan output

```
"halo"
```

nah php fpm itu kira kira bentuk lain dari itu yang kalau kita menjalankan di terminal itu kan kita menggunakan php nya langsung dan kalau kita menggunakan nginx kan otomatis nginx yang hanya tau ini file nya apa dan harus dikirim kemana kan tidak tau cara mengeksekusi file dengan ekstensi .php yang mana disini gunanya php fpm, jadi alurnya jika ada file .php pada direktori project php itu nginx hanya perlu tau kalau ada request kemana, harus kirim kemana dulu file nya biar setelah itu aku kirim lagi ke client yang mana beda dengan apache dengan mod_php nya yang mana itu adalah modul tambahan yang bisa dieksekusi oleh apache, ya bagusnya itu arsitektur yang sederhana tapi minusnya adalah apache akan tetap menggunakan itu even dia tidak melayani sesuatu yang menggunakan ekstensi .php yang itu hanya memberatkannya dan intinya dalah

- **Nginx + PHP-FPM**: dua proses dari container yang **terpisah**. Nginx meneruskan request `.php` lewat protokol FastCGI ke PHP-FPM yang berjalan mandiri di port 9000.
- **Apache + mod_php**: interpreter PHP **ditanam sebagai modul** di dalam proses Apache. Tidak ada proses kedua, tidak ada komunikasi jaringan internal.

Gambaran Alur (tambahan visual)

Display Mermaid diagrams in this vault?
Only allow if you trust this vault's contents.
Allow

```
flowchart LR
    C[Client] -->|HTTP| N[Nginx]
    N -->|FastCGI :9000| P[PHP-FPM]
    P -->|output| N
    N -->|response| C
```

Display Mermaid diagrams in this vault?
Only allow if you trust this vault's contents.
Allow

```
flowchart LR
    C2[Client] -->|HTTP| A["Apache + mod_php (satu proses)"]
    A -->|response| C2
```

Praktek Nginx + PHP-FPM

nah langsung aja kita langsung praktek dengan nginx + php fpm dulu yang langsung kita bikin dir baru dan langsung blasting aja dengan command

```
mkdir -p ~/web-php/html
```

next

```
cd ~/web-php
```

lalu

```
echo '<?php phpinfo();' > html/index.php
```

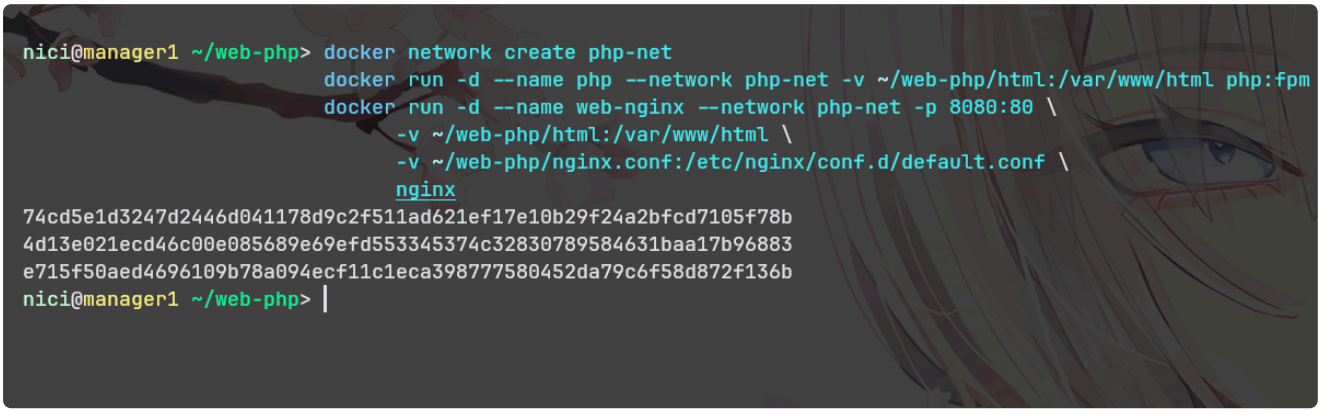
terus buat juga nginx.conf nya dengan isian

```
server {
    listen 80;
    root /var/www/html;
    index index.php index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_pass php:9000;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    }
}
```

terus jalankan dengan command



berikut adalah command command yang digunakan

```
docker network create php-net
```

disitu kita membuat sebuah network yang bernama php-net

```
docker run -d --name php --network php-net -v ~/web-php/html:/var/www/html
php:fpm
```

disitu kita menjalankan sebuah container yang bernama php dan disitu berisi php fpm untuk mengeksekusi kode php yang dikirim dari container nginx

```
docker run -d --name web-nginx --network php-net -p 8080:80 \
  -v ~/web-php/html:/var/www/html \
  -v ~/web-php/nginx.conf:/etc/nginx/conf.d/default.conf \
  nginx
```

disitu kita mengeksekusi container dengan nama web-nginx yang disitu kita menjalankan web server nya dengan menggunakan nginx. lalu kita akan coba curl aja

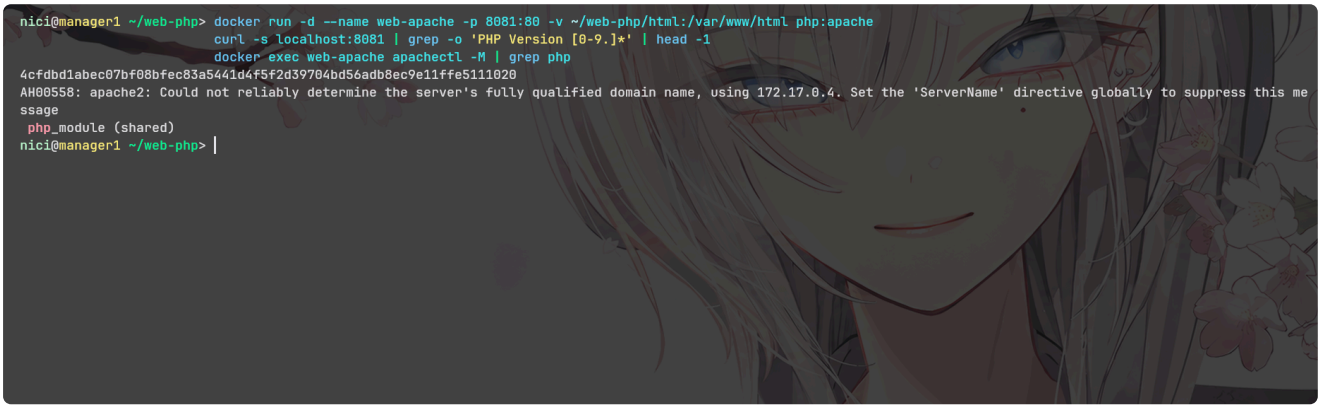
```
curl -s localhost:8080 | grep -o 'PHP Version [0-9.]*' | head -1
```



Praktek Apache + mod_php

terus lakukan hal yang sama dengan apache, langsung aja dengan command

```
docker run -d --name web-apache -p 8081:80 -v ~/web-php/html:/var/www/html
php:apache
curl -s localhost:8081 | grep -o 'PHP Version [0-9.]*' | head -1
docker exec web-apache apachectl -M | grep php
```



Docker Compose Dasar

lalu kita lanjut saja ke docker compose dasar yang dimana sebelumnya kita merasakan untuk membuat satu container nginx yang menjalankan php itu pun membutuhkan 2 container yang dijalankan satu satu dan docker compose ini sangat amat membantu karena kita hanya perlu untuk membuat satu file yaml dan mengeksekusi nya dan semua yang kita butuhkan akan jalan hanya dengan satu perintah, yang mana adalah

```
docker compose up -d
```

dan sebelum itu kita harus membuat sebuah file compose.yaml dengan command

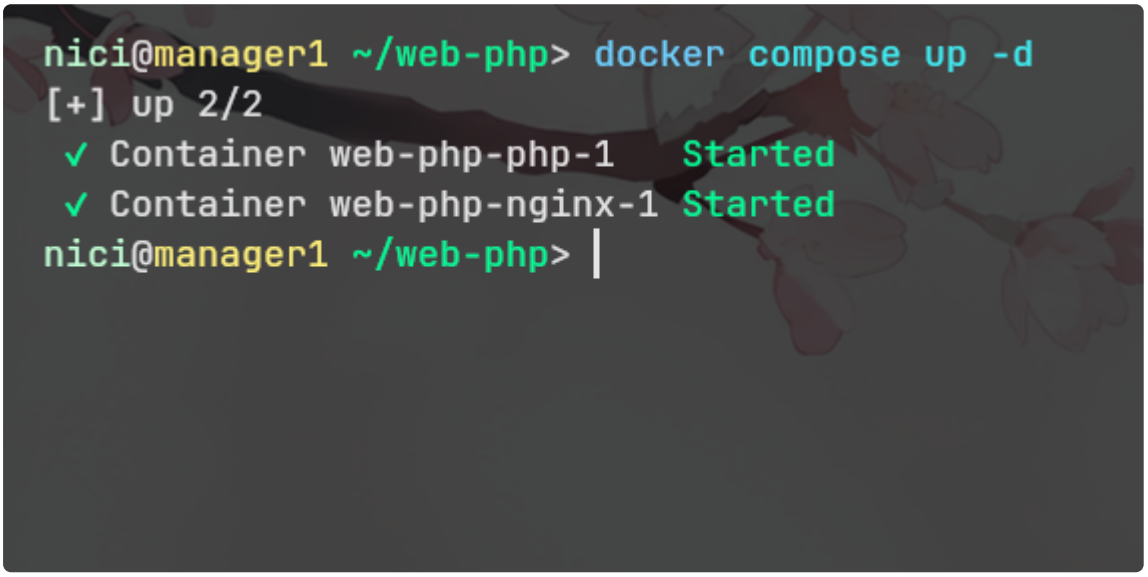
```
nano compose.yaml
```

lalu isi dengan

```
services:
  php:
    image: php:fpm
    volumes:
      - ./html:/var/www/html

  nginx:
    image: nginx
    ports:
      - "8080:80"
    volumes:
      - ./html:/var/www/html
      - ./nginx.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - php
```

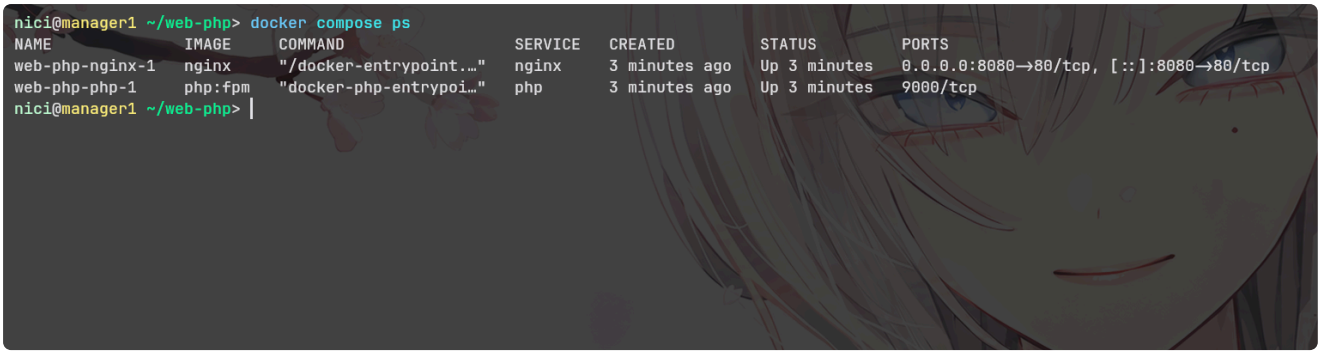
langsung aja kita jalankan yang mana hasilnya kira kira seperti ini



bisa kita langsung dengan

```
docker compose ps
```

yang hasilnya adalah



Deploy Aplikasi PHP dengan Nginx dan PHP-FPM via Compose

lalu kita akan langsung aja lanjut ke Deploy aplikasi PHP (Laravel, CodeIgniter, atau WordPress) dengan Nginx dan PHP-FPM via Compose. dan disini kita akan membuat wordpress karena wordpress itu paling simpel dan udah ada image yang tinggal diinstall pake composer install aja dan kita tinggal membuat direktori baru dan compose baru dengan command

```
mkdir wordpress-deploy
```

lalu

```
cd wordpress-deploy
```

terus buat dulu konfigurasi nginx nya dengan command

```
nano nginx.conf
```

yang berisi

```
server {
    listen 80;
    root /var/www/html;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$args;
```

```
    }

    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_pass wordpress:9000;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    }
}
```

> 🔥 **Beda dari Config PHP Sebelumnya yang jadi beda dari config PHP yang kemarin cuma di baris `try_files /index.php?$args` itu yang bikin URL kayak `/tentang-kami` tetap diarahkan ke `index.php`, biar WordPress yang urus routing-nya sendiri (ini yang disebut "pretty permalink"),**

next kita tinggal buat compose nya dengan

```
nano compose.yaml
```

```
services:
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: rootpass123
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass123
    volumes:
      - db-data:/var/lib/mysql

  wordpress:
    image: wordpress:fpm
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass123
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp-content:/var/www/html
    depends_on:
      - db

  nginx:
    image: nginx
    ports:
      - "8090:80"
    volumes:
      - wp-content:/var/www/html
```

```
      - ./nginx.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - wordpress

volumes:
  db-data:
  wp-content:
```

> ⚠ **Catatan Keamanan (tambahan) Password di atas hanya untuk lab. Jangan dipakai di lingkungan produksi.**

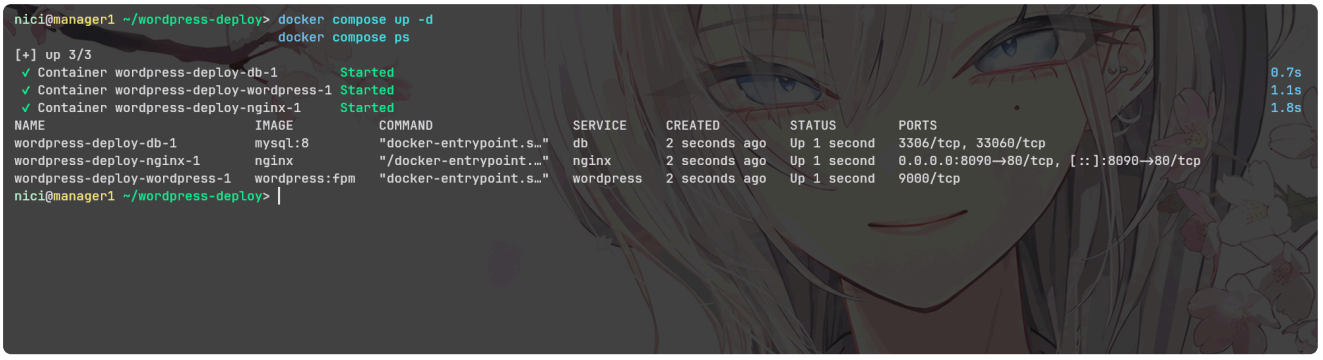
lalu setelah membuat compose nya, kita tinggal hajar pake command

```
docker compose up -d
docker compose ps
```

lalu tinggal lihat aja dengan

```
docker compose ps
```

yang hasilnya seperti ini



Uji Beban Apache dan Nginx

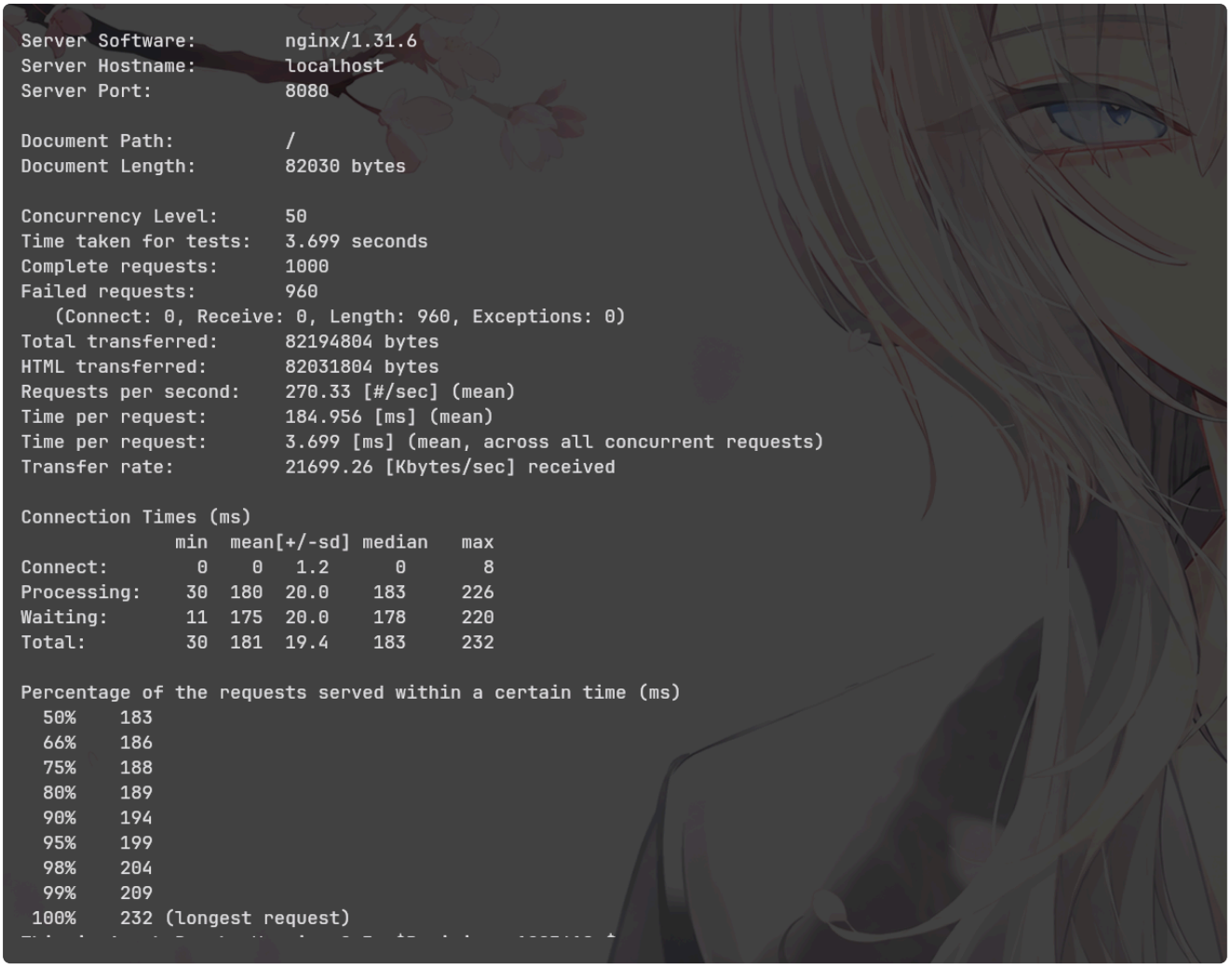
next kita tinggal uji beban apache dan nginx dengan menggunakan tools apache-utilss dan langsung aja kita eksekusi dengan command

```
sudo apt install apache2-utils -y
```

lalu langsung aja kita tes kriuk dengan command

```
ab -n 1000 -c 50 -H "Host: site-a.local" http://localhost:8080/   # Nginx
ab -n 1000 -c 50 -H "Host: site-a.local" http://localhost:8081/   # Apache
```

dan hasilnya seperti ini untuk nginx



```
Server Software:        nginx/1.31.6
Server Hostname:        localhost
Server Port:            8080

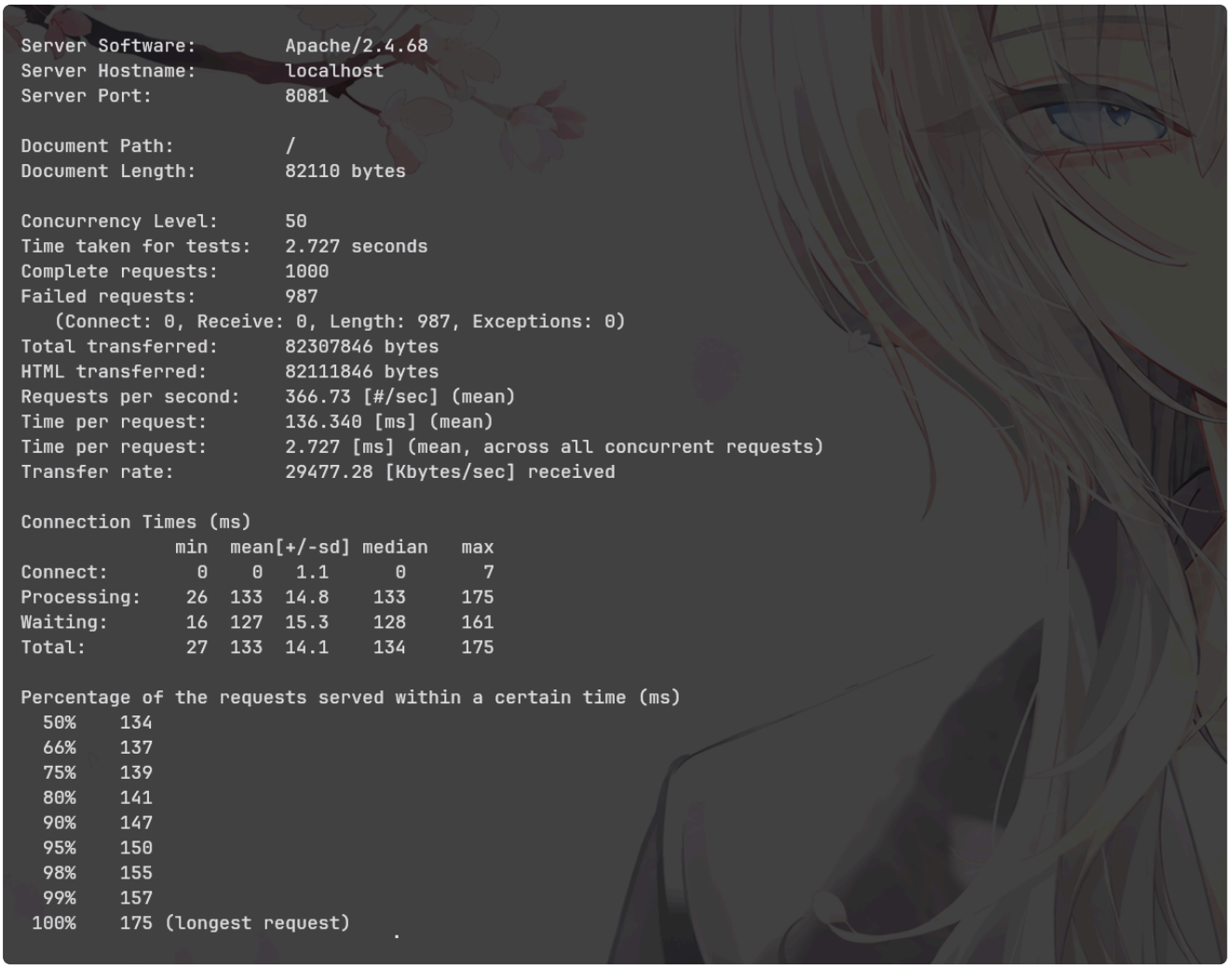
Document Path:          /
Document Length:        82030 bytes

Concurrency Level:      50
Time taken for tests:   3.699 seconds
Complete requests:      1000
Failed requests:        960
   (Connect: 0, Receive: 0, Length: 960, Exceptions: 0)
Total transferred:      82194804 bytes
HTML transferred:       82031804 bytes
Requests per second:    270.33 [#/sec] (mean)
Time per request:       184.956 [ms] (mean)
Time per request:       3.699 [ms] (mean, across all concurrent requests)
Transfer rate:          21699.26 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   1.2      0       8
Processing:    30  180  20.0    183     226
Waiting:       11  175  20.0    178     220
Total:         30  181  19.4    183     232

Percentage of the requests served within a certain time (ms)
  50%    183
  66%    186
  75%    188
  80%    189
  90%    194
  95%    199
  98%    204
  99%    209
 100%    232 (longest request)
```

ini apache



yang mana kira kira summary nya gini mas

Metrik	Nginx (n=1000, c=50)	Apache (n=1000, c=50)	Nginx (n=5000, c=200)	Apache (n=5000, c=200)
Requests/sec	**2173.84**	1262.93	**2069.53**	1501.85
Time per request (mean)	**23.0 ms**	39.6 ms	**96.6 ms**	133.2 ms
Failed requests	0	0	0	0
99th percentile	**39 ms**	111 ms	**148 ms**	485 ms
Max (100%)	**39 ms**	135 ms	**161 ms**	707 ms

Hardening Nginx

lalu tinggal ke hardening nya yang intinya itu adalah kita membatasi informasi dan attack vector yang akan didapat oleh attacker tanpa usaha yang gimana gimana, sebenernya simpel yang mana kita hanya perlu untuk sedikit memodifikasi konfigurasi nginx.conf yang sebelumnya seperti ini,

```
server {
    listen 80;
    server_name site-a.local;
    root /sites/site-a;
    index index.html;
}

server {
    listen 80;
    server_name site-b.local;
    root /sites/site-b;
    index index.html;
}
```

jadi seperti ini

```
server_tokens off;

server {
    listen 80;
    server_name site-a.local;
    root /sites/site-a;
    index index.html;
    autoindex off;

    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header Content-Security-Policy "default-src 'self'" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
}
```

dan penjelasan simpelnya disitu kita menambahkan:

Penjelasan Tiap Direktif

- `server_tokens off;` untuk hiding versi nginx nya
- `autoindex off;` untuk mitigasi directory listing
- `X-Content-Type-Options "nosniff"` buat maksa browser itu menyesuaikan conten-type yang akan dikirim ke server, agar gaada orang orang nakal yang menggunakan sesuatu seperti .img.php
- `X-Frame-Options "SAMEORIGIN"` ini untuk mencegah adanya clickjacking
- `Content-Security-Policy "default-src 'self'"` buat batesin sumber resource dari domain kita sendiri buat mitigasi xss

- `Referrer-Policy "strict-origin-when-cross-origin"` membatasi informasi soal url yang dikirim ke web lain saat ngeklik link keluar

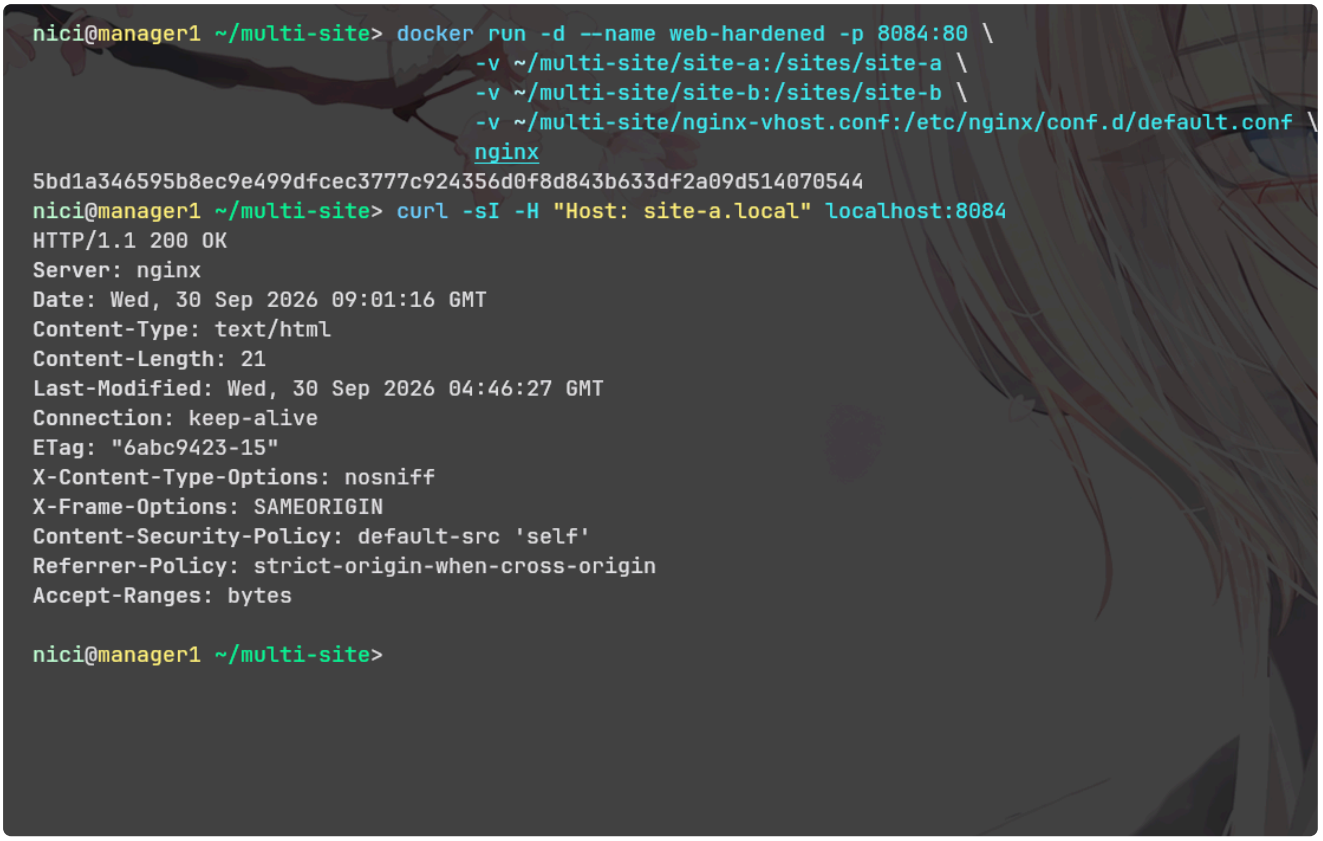
langsung aja kita tes kriuk dengan command

```
docker run -d --name web-hardened -p 8084:80 \
  -v ~/multi-site/site-a:/sites/site-a \
  -v ~/multi-site/site-b:/sites/site-b \
  -v ~/multi-site/nginx-vhost.conf:/etc/nginx/conf.d/default.conf \
  nginx
```

lalu verif dengan command

```
curl -sI -H "Host: site-a.local" localhost:8084
```

terus cek yang hasilnya kaya gini



terus hasilnya kira kira kaya gitu, apa yang kita taruh di nginx.conf nya keluar semua kan.

TLS dengan CA Lokal

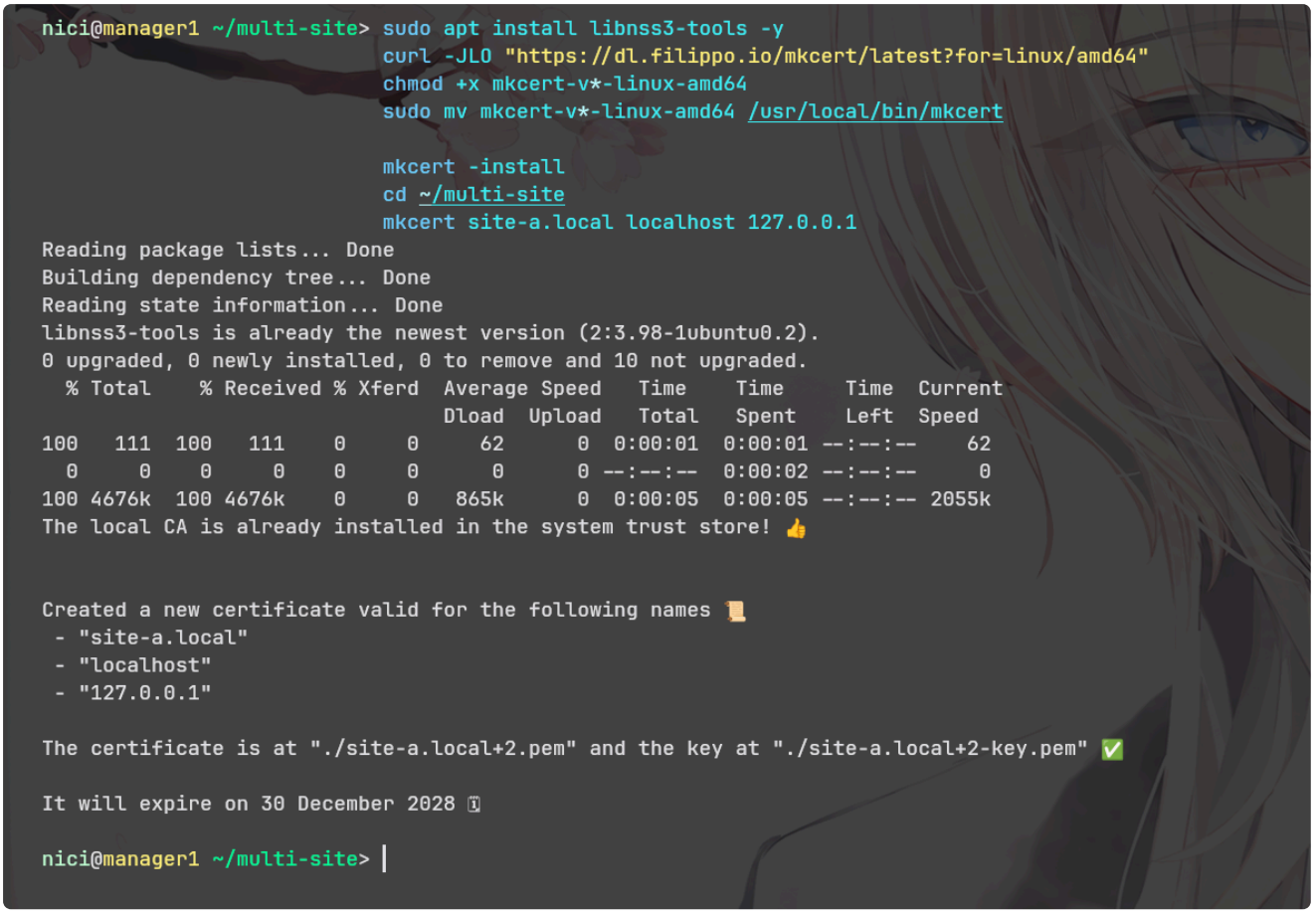
dan yang terakhir adalah tls dengan ca lokal yang mana ini intinya kita menggunakan mkcert untuk membuat ca certificate yang akan kita daftarkan ke trust store system yang menjadi

perbedaan dari kita memasukkan ca cert sendiri itu adalah ca certificate yang ada kita buat pada dasarnya ca kita emang sudah dipercaya secara eksplisit karena kita kan itu membuat sendiri dan tools yang dipakai juga sudah trusted. dan disini kita langsung aja ke bagian step by step untuk instalsi dan pembuatan sertifikat, kita akan gunakan command command sebagai berikut

```
sudo apt install libnss3-tools -y
curl -JLO "https://dl.filippo.io/mkcert/latest?for=linux/amd64"
chmod +x mkcert-v*-linux-amd64
sudo mv mkcert-v*-linux-amd64 /usr/local/bin/mkcert

mkcert -install
cd ~/multi-site
mkcert site-a.local localhost 127.0.0.1
```

yang mana kira kira hasilnya kaya gini



pada hasil diatas itu yang bagian mkcert --install itu kita membuat key yang mana itu sebagai ca cert baru dan satu lagi kita sekalian mendaftarkan ca cert yang telah kita buat ke trust store sistem kita dan pada command

```
mkcert site-a.local localhost 127.0.0.1
```

disitu kita nerbitin sertifikat untuk 3 nama sekaligus, yaitu site-a.local, localhost, dan 127.0.0.1(ip loopback kita) dan terus kita akan membuat konfigurasi nginx untuk ini agar

koneksi kita menggunakan protokol https yang berjalan pada port 443, bukan http yang berjalan pada port 80, langsung aja kita buat dengan command

```
nano nginx-tls.conf
```

yang berisi

```
server {
    listen 443 ssl;
    server_name site-a.local;
    root /sites/site-a;
    index index.html;

    ssl_certificate /certs/site-a.local+2.pem;
    ssl_certificate_key /certs/site-a.local+2-key.pem;

    add_header Strict-Transport-Security "max-age=31536000" always;
}
```

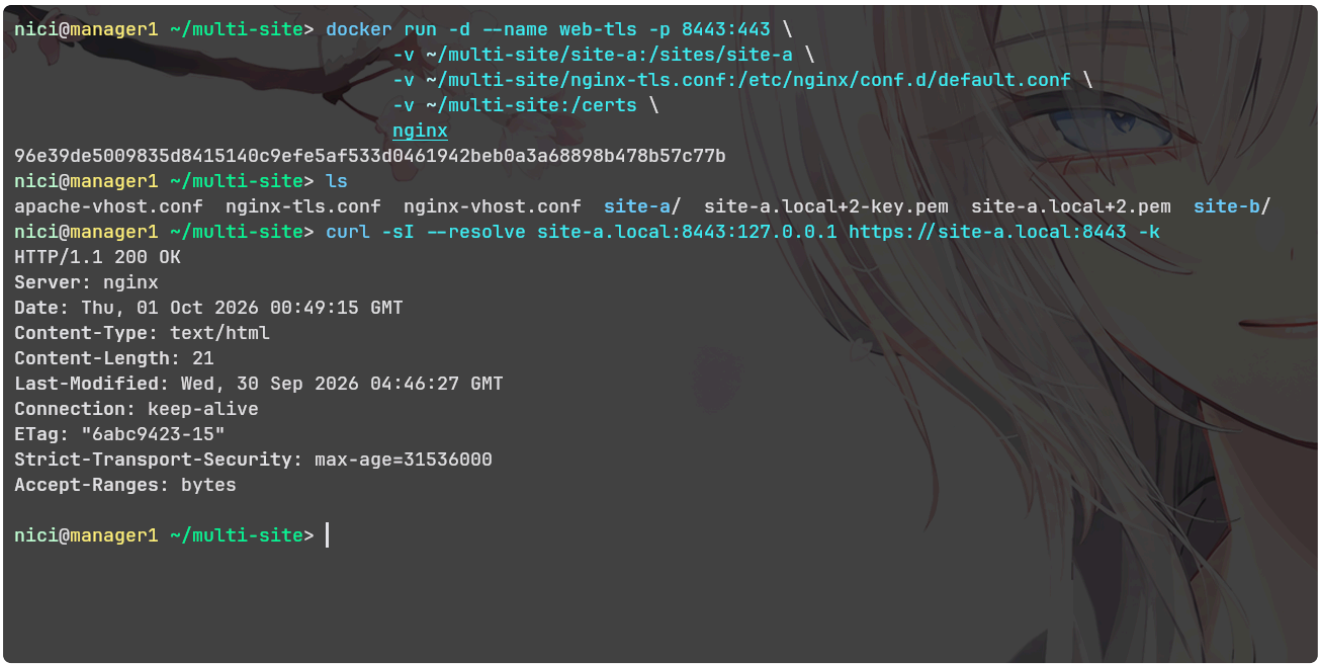
setelah itu langsung jalankan dengan command

```
docker run -d --name web-tls -p 8443:443 \
  -v ~/multi-site/site-a:/sites/site-a \
  -v ~/multi-site/nginx-tls.conf:/etc/nginx/conf.d/default.conf \
  -v ~/multi-site:/certs \
  nginx
```

terus langsung aja kita verif dengan command

```
curl -sI --resolve site-a.local:8443:127.0.0.1 https://site-a.local:8443 -
k
```

yang hasilnya kira kira seperti ini



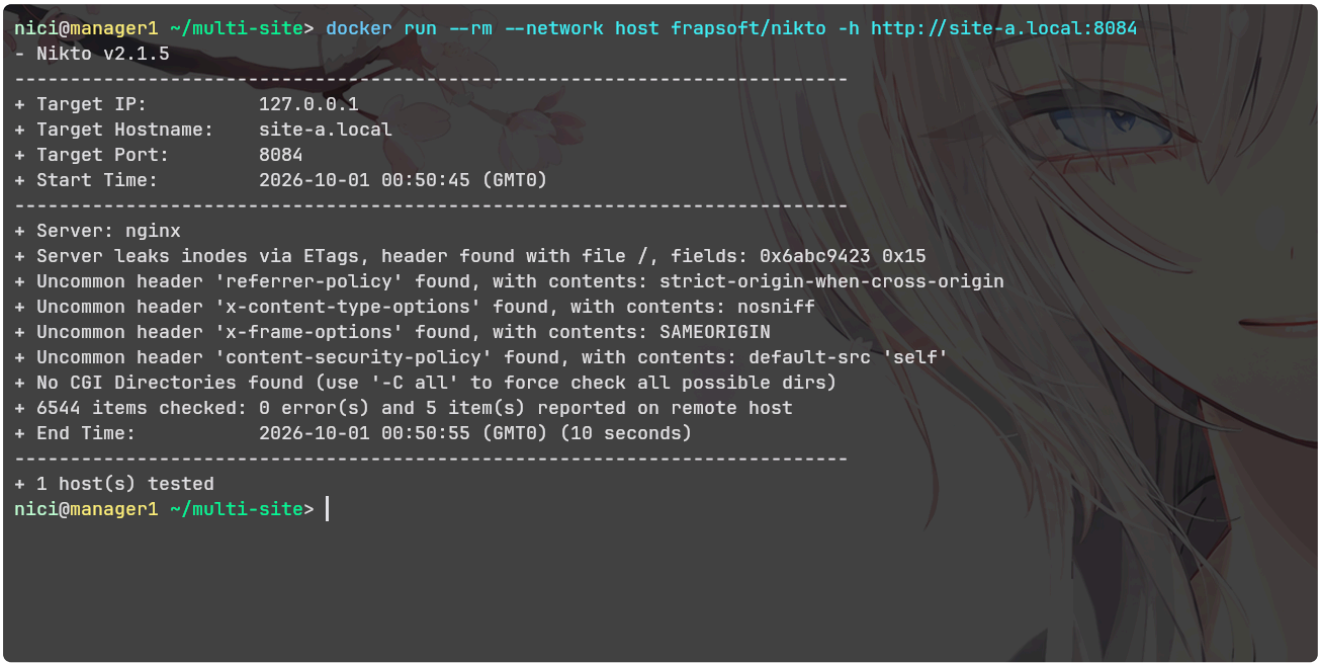
Scan Keamanan

Nikto

dan untuk sentuhan terakhir, mari kita langsung aja kita sekalian untuk scan dengan menggunakan nikto untuk keamanan dari container kita langsung dengan command

```
docker run --rm --network host frapsoft/nikto -h http://site-a.local:8084
```

yang hasilnya kria kira seperti ini



nah uji dari nikto sudah selesai yang mana rangkuman yang ditemukan seperti ini

```
6544 items checked: 0 error(s) and 5 item(s) reported
```

terus yang bagian uncommon hanyalah warning karena header dll kita itu menggunakan header yang uncommon dan ini hanyalah sebuah warning dan bagian etag itu ke leaks karena aku tidak menambahkan fungsin untuk mematikan itu jadi kita bisa tambahkan bari

```
etag off;
```

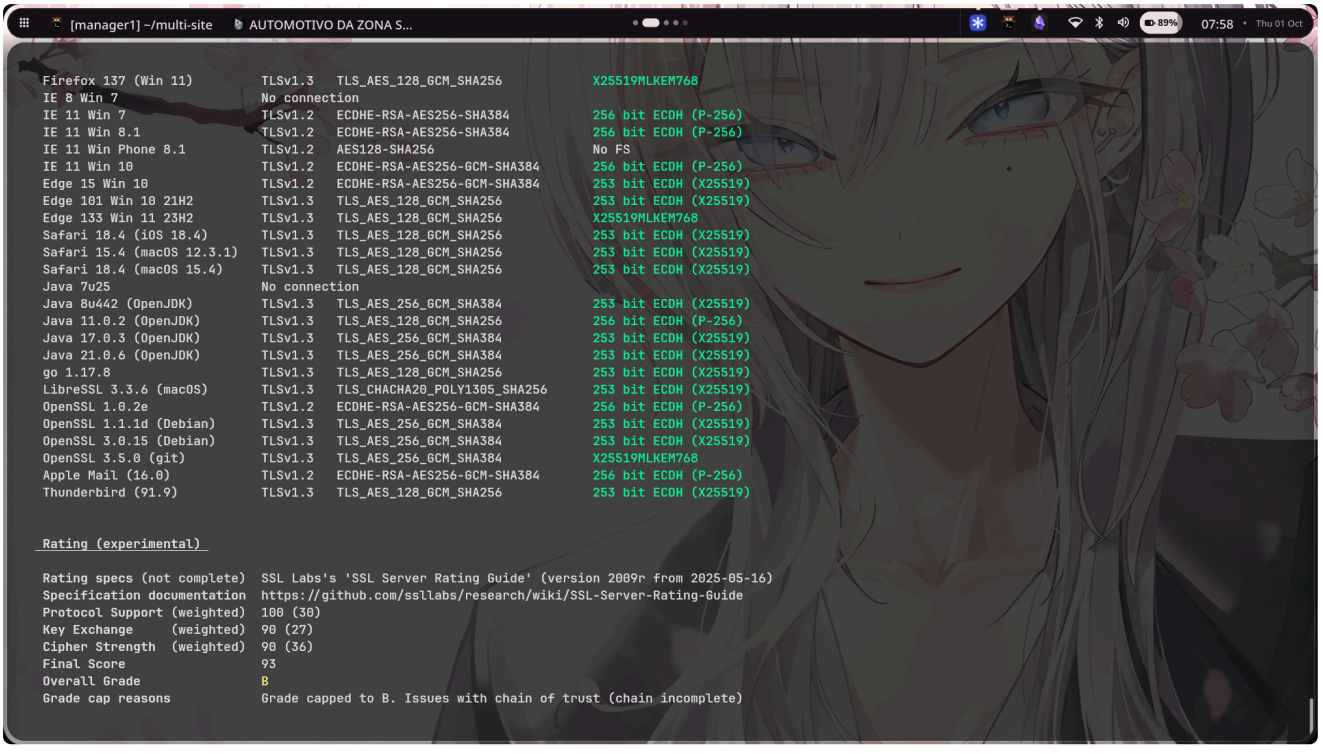
pada konfigurasi nginx-tls.conf nya

testssl.sh

dan yg terakhir banget ini kita sekalian uji saja dengan menggunakan testssl.sh untuk audit konfigurasi ssl pada server dan langsung aja kita akan menggunakan command

```
docker run --rm -ti --network host drwetter/testssl.sh https://site-a.local:8443
```

yang hasilnya seperti ini



terlalu banyak kan? intinya ringkasannya seperti ini

Protokol	Status
SSLv2, SSLv3	not offered (OK)
TLS 1.0, TLS 1.1	not offered (baik — protokol usang dinonaktifkan)
TLS 1.2	offered (OK)

Protokol	Status
TLS 1.3	offered (OK)

Tabel Port yang Dipakai (tambahan)

Port Host	Container	Keterangan
8080	`web-nginx`	Nginx + PHP-FPM
8081	`web-apache`	Apache + mod_php
8082	`web-nginx-vhost`	Server block Nginx
8083	`web-apache-vhost`	Virtual host Apache
8084	`web-hardened`	Nginx hardening
8090	`nginx` (compose)	WordPress
8443	`web-tls`	Nginx TLS (mkcert)

---

oke sekian dari dokumentasiku, makasih!